

EC2에 Jenkins로 CI/CD (완)

과정

1. EC2 인스턴스에 Jenkins 컨테이너 띄우기
2. Jenkins 설정
3. Jenkins와 GitLab 연동
4. GitLab 저장소 변경 감지 및 Jenkins 빌드 트리거
5. Jenkins에서 코드 빌드 및 Docker 이미지 생성 및 컨테이너 생성

GitLab 연동

1. Jenkins에 GitLab 플러그인 설치
2. GitLab API 토큰 생성 및 Jenkins Credentials에 등록

CI/CD 파이프라인 구성

1. Jenkins 프로젝트 생성 및 GitLab 저장소 연결
2. Jenkinsfile 작성:
 - 코드 체크아웃
 - 빌드 스크립트 실행
 - Docker 이미지 빌드
 - Docker Compose 파일 생성 또는 업데이트
 - Docker Compose를 사용하여 새 컨테이너 실행
3. GitLab Webhook 설정으로 코드 변경 시 자동 빌드 트리거

이 설정을 통해 GitLab 저장소에 변경사항이 push되면 자동으로 Jenkins 빌드가 시작되고, 새로운 Docker 이미지가 생성되어 EC2 인스턴스에서 실행됩니다.

EC2에 Docker로 Jenkins 컨테이너 띄우기

우리는 EC2 환경에서 Jenkins 컨테이너를 생성하여, CI/CD를 할 계획이다.

왜냐하면, 로컬에서 Jenkins를 실행하면 인터넷이 끊기거나 해당 개발자의 PC가 꺼지면 CI/CD가 중단 되기 때문이다.

젠킨스 설치 및 실행

```
docker run -d \
  --name jenkins \
  --group-add $(stat -c '%g' /var/run/docker.sock) \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v jenkins_home:/var/jenkins_home \
  -p 8080:8080 \
  -p 50000:50000 \
  jenkins/jenkins:its
```

--group-add \$(stat -c '%g' /var/run/docker.sock)

- Jenkins 컨테이너가 **Docker 데몬**(`docker.sock`)을 제어할 수 있도록 권한을 추가
- **이유:** Jenkins가 Docker 명령어(`docker build` , `docker run` 등)를 실행하려면 **호스트의 Docker 소켓**(`/var/run/docker.sock`)에 접근할 수 있어야 함
- `$(stat -c '%g' /var/run/docker.sock)` → **Docker 소켓 파일의 그룹 ID를 가져와서 Jenkins 컨테이너에 추가**
- 즉, Jenkins 컨테이너 내부에서 Docker 명령어를 실행할 수 있도록 설정

-v jenkins_home:/var/jenkins_home

- Docker 데몬과 Jenkins 컨테이너가 직접 통신할 수 있도록 마운트
- Jenkins 내부에서 `docker build` , `docker run` 같은 명령을 실행할 수 있게 해줌
- 즉, Jenkins 컨테이너 안에서 Docker 컨테이너를 실행 가능 (`Docker in Docker` 방식)
- 그러므로 Jenkins에서 이미지를 만들고, 실행을 하면 EC2 서버에서 컨테이너를 띄울 수 있음.

-v jenkins_home:/var/jenkins_home

- Jenkins의 데이터(플러그인, 설정, 빌드 기록 등)를 저장하는 볼륨
- 컨테이너를 삭제해도 Jenkins 데이터를 유지하려면 볼륨(`jenkins_home`)을 사용해야 함
- `jenkins_home` 볼륨이 `/var/jenkins_home` 디렉토리와 연결됨
- Jenkins가 처음 실행될 때 기본적으로 `/var/jenkins_home` 에 데이터를 저장함

포트 번호

p 8080:8080 → Jenkins 웹 UI 접속용

- **8080** 포트는 **Jenkins 관리 웹페이지 (Web UI)** 에 접속하는 데 사용됨.
- Jenkins를 실행한 후, 웹 브라우저에서 `http://[자기 도메인]:8080` 로 접속하면 Jenkins 대시보드를 볼 수 있음

p 50000:50000 → Jenkins Master-Agent 통신용

- Jenkins는 **마스터-에이전트(Master-Agent) 아키텍처**를 지원함.
- 에이전트(Node)는 **Jenkins Master**에서 실행된 빌드를 처리하는 별도의 노드(서버)임.
- **50000** 포트는 **Jenkins Master와 Agent 간의 통신에 사용됨.**
 - Master → Agent로 빌드 작업을 전달
 - Agent → Master로 빌드 실행 결과를 반환
- 만약 Jenkins를 **단일 서버에서만 실행하고, 추가 에이전트를 사용하지 않는다면 이 포트는 필요 없음.**(즉, **p 50000:50000** 은 선택사항)

즉, **50000** 포트는 **Jenkins Master와 Agent 간의 통신을 위해 사용됨.**

단일 EC2 인스턴스에서만 Jenkins를 사용할 경우, 이 포트는 생략 가능함.

Jenkins 설정

1. Jenkins 컨테이너의 외부포트가 8080 이기 때문에, `http://[자기 도메인]:8080` 으로 들어가면, 아래처럼 나오는데

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

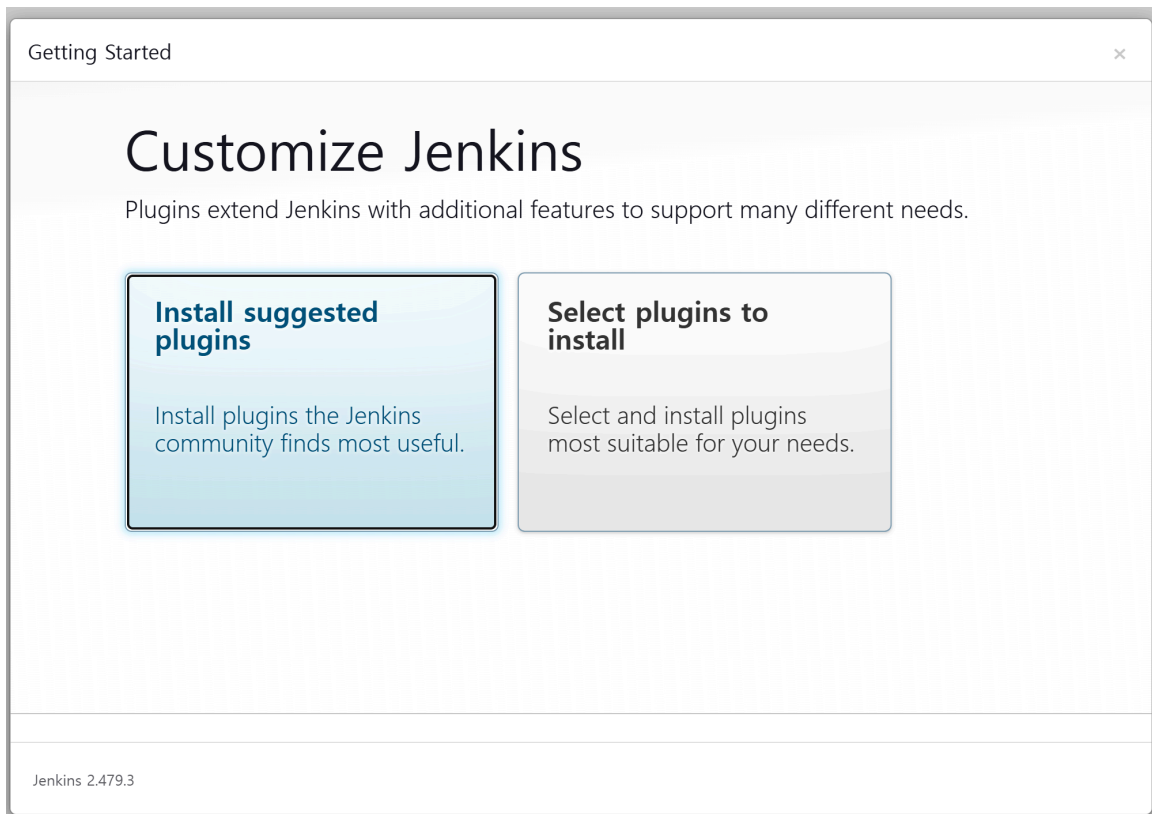
Continue

Administrator password는 Jenkins 컨테이너를 설치한 서버에서 아래와 같은 명령어를 치면 나온다.

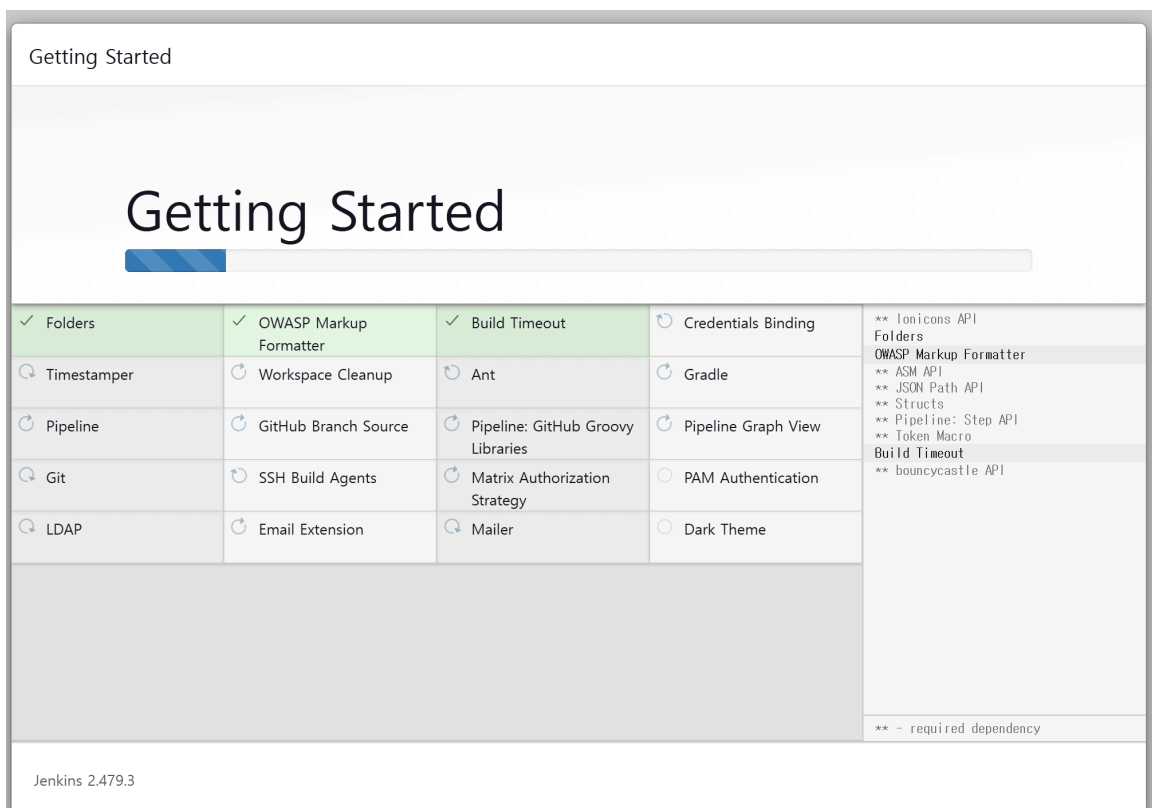
```
# 명령어
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
# 결과
1e9986d7b52743fbb2b072a303927684
```

Administrator password 에 나온 비밀번호를 치고 Continue 버튼 클릭!

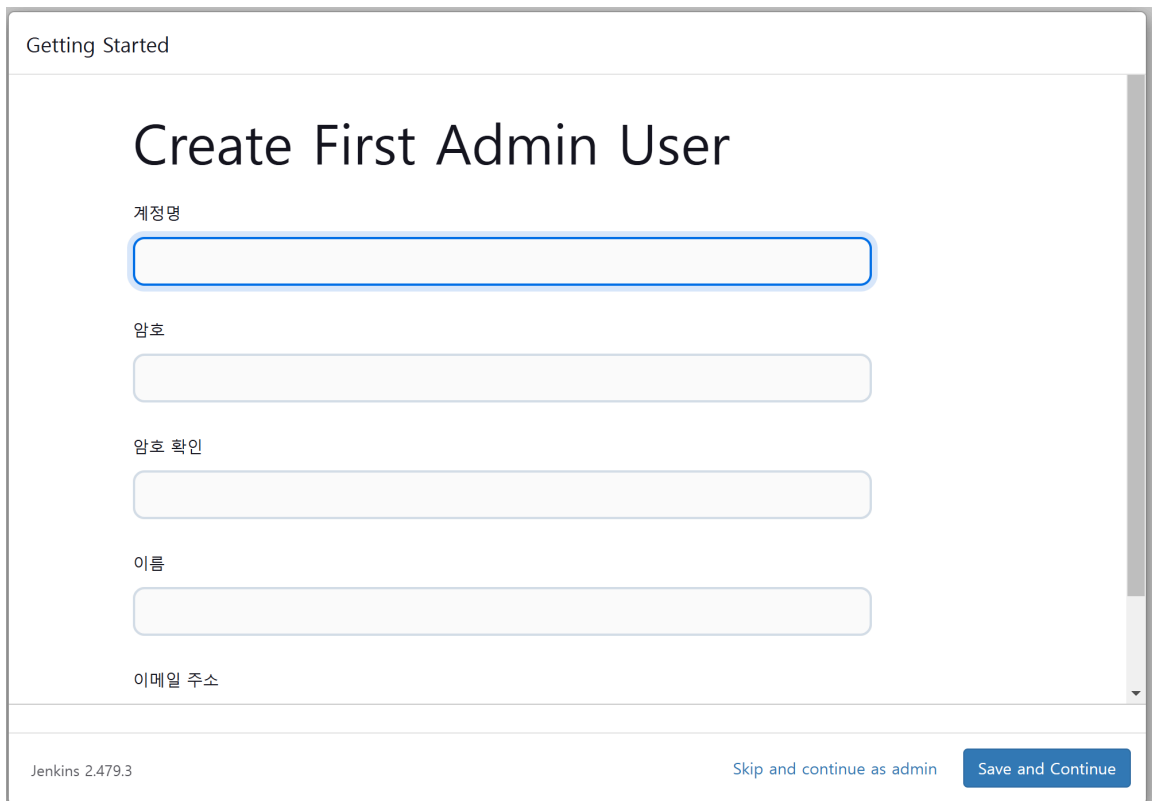
- 그럼 아래와 같은 웹 페이지가 나오는데 왼쪽 Install suggested plugins 버튼을 클릭한다.



그러면 아래와 같은 화면으로 설치가 진행된다.



설치가 완료되면 계정 정보를 작성하면 된다. 젠킨스 웹 페이지에 접속할 때 필요하므로 기억해야 된다.



The image shows the 'Getting Started' page of Jenkins 2.479.3. The main heading is 'Create First Admin User'. Below it, there are five input fields with labels in Korean: '계정명' (Username), '암호' (Password), '암호 확인' (Confirm Password), '이름' (Name), and '이메일 주소' (Email address). At the bottom left, it says 'Jenkins 2.479.3'. At the bottom right, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

Save and Continue 를 누르면 아래와 같은 화면이 나타나는데, Jenkins 컨테이너에 접속할 URL이 보인다.

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the `BUILD_URL` environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.479.3

Not now [Save and Finish](#)

현재는 로컬에서 새 컨테이너를 만들어 새로 시작하기 때문에 도메인은 localhost로 되어있고 포트번호는 9090 으로 되어있다.

Save and Fininsh 를 누르자

Getting Started

Jenkins is ready!

Your Jenkins setup is complete.

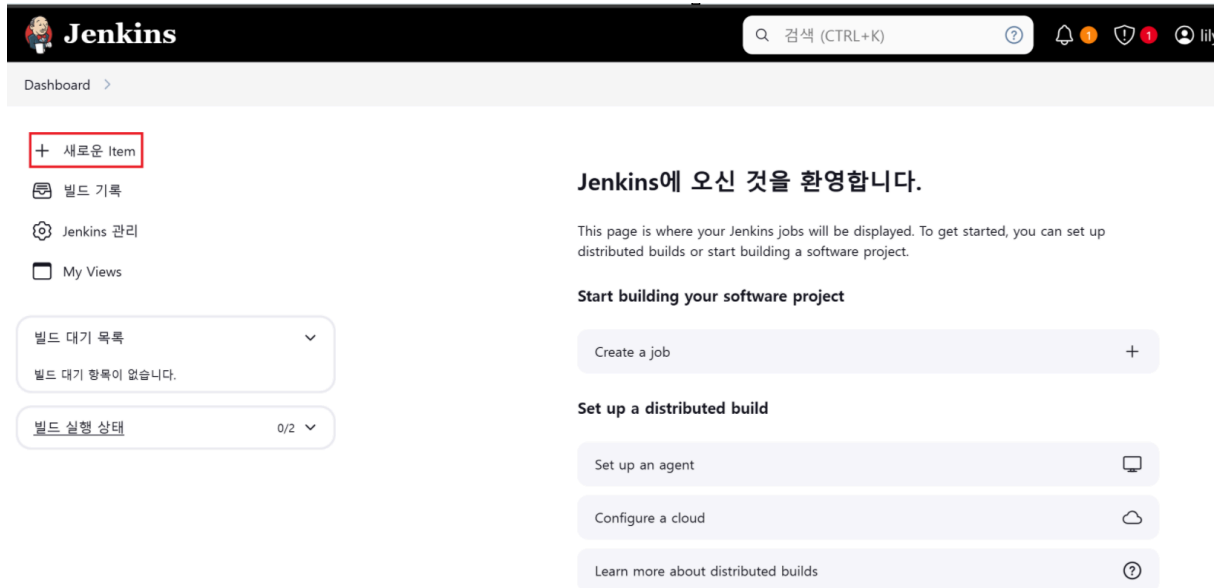
[Start using Jenkins](#)

Jenkins 2.479.3

Start using Jenkins 를 클릭하면 Jenkins 웹 페이지가 열린다.

Jenkins와 GitLab 연동 (수정 필요)

1. Jenkins에서 새로운 Item 을 누른다.



2. 그러면 다음과 같은 창이 뜨는데, Item 이름을 적고, Freestyle project를 클릭하고, OK를 누른다.

 Jenkins

검색 (CTRL+K)

Dashboard > All > New Item

New Item

Enter an item name

» This field cannot be empty, please enter a valid name

Select an item type

 Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

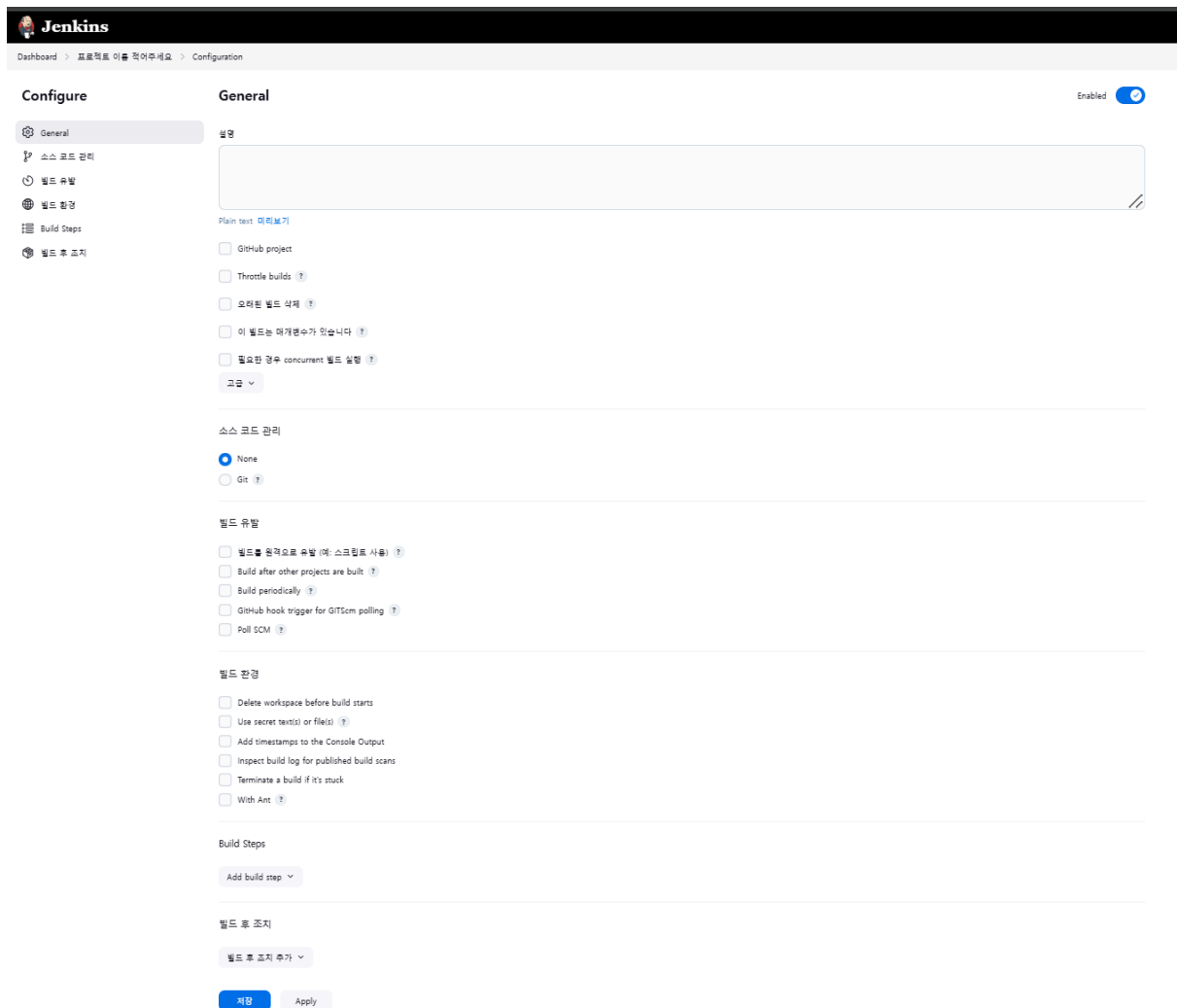
 Multi-configuration project
다양한 환경에서의 테스트, 플래폼 특성 빌드, 기타 등등 처럼 다수의 서로다른 환경설정이 필요한 프로젝트에 적합함.

 Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 Multibranch Pipeline

OK

3. 그러면 다음과 같은 창이 뜨는데, 여기에서는 깃랩과 연동을 하기 위한 설정을 해주자.



The image shows the Jenkins 'Configure' page for a new item. The 'General' tab is selected in the left sidebar. The main content area is titled 'General' and includes an 'Enabled' toggle switch. The 'Description' field is empty. Under 'Plain text editor', there are several checkboxes: 'GitHub project', 'Throttle builds', 'Optimize build queue', 'This build is part of a larger build', and 'Require concurrent build'. The 'Source Code Management' section has 'None' selected. The 'Build Triggers' section has several checkboxes: 'Build when a change is pushed to the branch', 'Build after other projects are built', 'Build periodically', 'GitHub hook trigger for GITSCM polling', and 'Poll SCM'. The 'Build Environment' section has checkboxes for 'Delete workspace before build starts', 'Use secret text(s) or file(s)', 'Add timestamps to the console output', 'Inspect build log for published build scans', 'Terminate a build if it's stuck', and 'With Ant'. The 'Build Steps' section has an 'Add build step' button. The 'Post-build Actions' section has a 'Add post-build action' button. At the bottom, there are 'Save' and 'Apply' buttons.

4. 깃랩과 연동하기 위해 먼저 체크부터 하자.

- 설명에 이 Item의 용도 적기 (ex : Jenkins CI/CD)

설명

- 소스 코드에 Git 클릭

- CI/CD 하고싶은 Git의 Repository URL 적기 (ex: `https://testProject/project.git`)

오류가 뜨는 이유는 Credentials 설정을 안해서 그렇다.

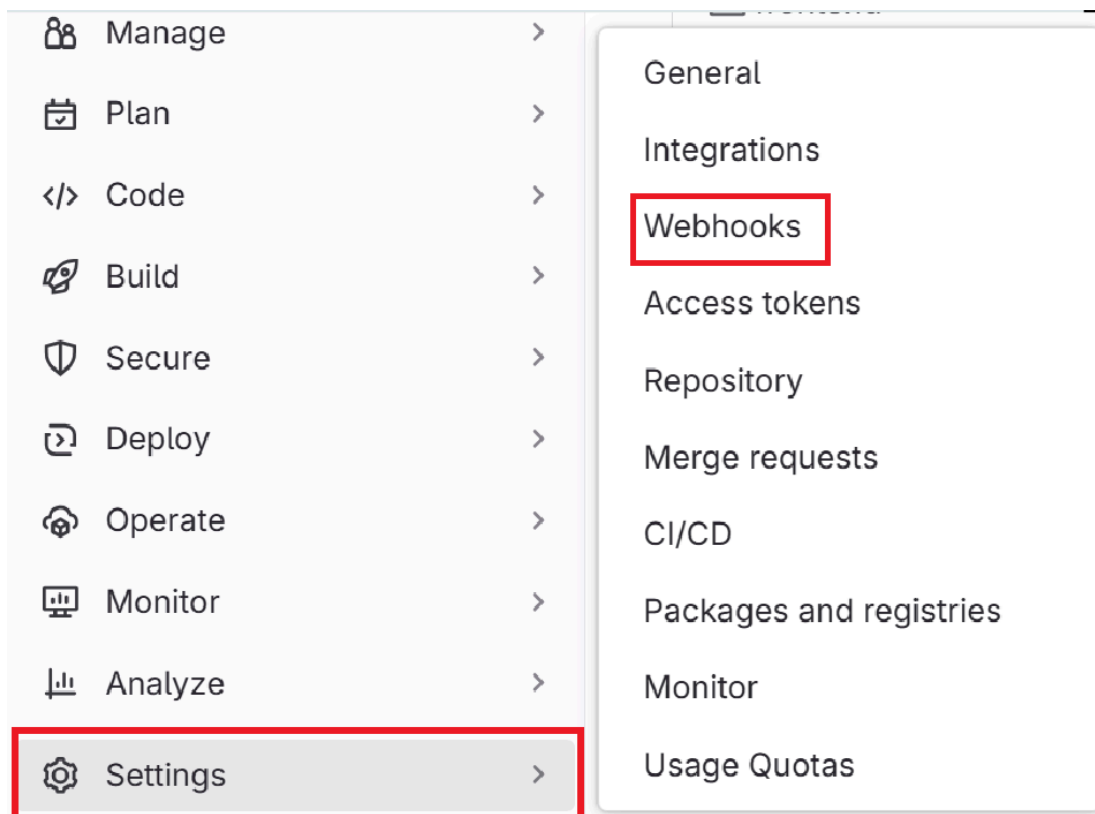
- Credentials 설정
 - Add → Jenkins 클릭하면 아래와 같은 창이 뜬다.

- 여기서 Username은 깃랩 아이디를 적고,
- password는 무조건 깃랩 Personal AccessToken을 적는다.

- ID는 Credentials 이름을 적는다. (즉, Credentials 를 식별하는 고유 ID)
- 잘 입력하고, 만들어진 Credentials를 클릭하면 아래와 같이 오류가 사라진다.

The screenshot shows the 'Repositories' section in GitLab. It contains a 'Repository URL' field with the value 'https://lab.ssafy.com/s12-webmobile2-sub1/S12P11B108.git' and a 'Credentials' dropdown menu with the selected value 'thddydds1@naver.com/***** (Test Jenkins입니다.)'. Below these fields are '+ Add' and '고급' (Advanced) buttons.

- GitLab에서 코드 변경 시 자동으로 Jenkins가 빌드를 트리거하도록 Webhook을 설정해야 한다. 깃랩 프로젝트에서 Settings → Webhook 으로 이동하자.



그리고 Add new Webhook을 누르자



그러면 아래와 같은 창이 뜨고, 설정을 하면된다. access token은 jenkins와 gitlab을 연결 해 줄 access token 이다.

그리고 Trigger에서는 CI/CD를 할 브랜치 명을 적어주면 된다.

Webhooks

Webhooks enable you to send notifications to web applications in response to events in a group or project. We recommend using an [integration](#) in preference to a webhook.

Webhooks 4

URL

http://localhost/project/전킨스 item 이



Secret token will be cleared on save unless token is updated.

URL must be percent-encoded if it contains one or more special characters.

☒ Show full URL

☐ Mask portions of URL

Do not show sensitive data such as tokens in the UI.

Custom headers </> 0

Add custom header

No custom headers configured.

Name (optional)

Description (optional)

Secret token

Used to validate received payloads. Sent with the request in the `X-GitLab-Token` HTTP header.

Trigger

☒ Push events

☐ All branches

☒ Wildcard pattern

develop

Wildcards such as `*-stable` or `production/*` are supported.

☐ Regular expression

☐ Tag push events

A new tag is pushed to the repository.

☐ Comments

A comment is made or edited on an issue or merge request.

☐ Confidential comments

A comment is made or edited on a confidential issue.

☐ Issues events

An issue is created, updated, closed, or reopened.

☐ Confidential issues events

A confidential issue is created, updated, closed, or reopened.

☒ Merge request events

A merge request is created, updated, or merged.

☐ Job events

A job's status changes.

☐ Pipeline events

A pipeline's status changes.

☐ Wiki page events

A wiki page is created or updated.

☐ Deployment events

A deployment starts, finishes, fails, or is canceled.

☐ Feature flag events

A feature flag is turned on or off.

☐ Releases events

A release is created, updated, or deleted.

☐ Emoji events

An emoji is awarded or revoked. Which emoji events trigger webhooks?

☐ Project or group access token events

An access token is going to expire in the next 7 days. Which project or group access token events trigger webhooks?

Custom webhook template (optional)

[How to create a custom webhook template?](#)

SSL verification

☒ Enable SSL verification

Add webhook

Cancel

그리고 Add webhook을 누르면 webhook이 생성된다.

- CI/CD 를 하고 싶은 Git Branch 설정 (dev 브랜치에 있는 프로젝트로 CI/CD 하고 싶으면 */dev 로 입력)

Branches to build ?

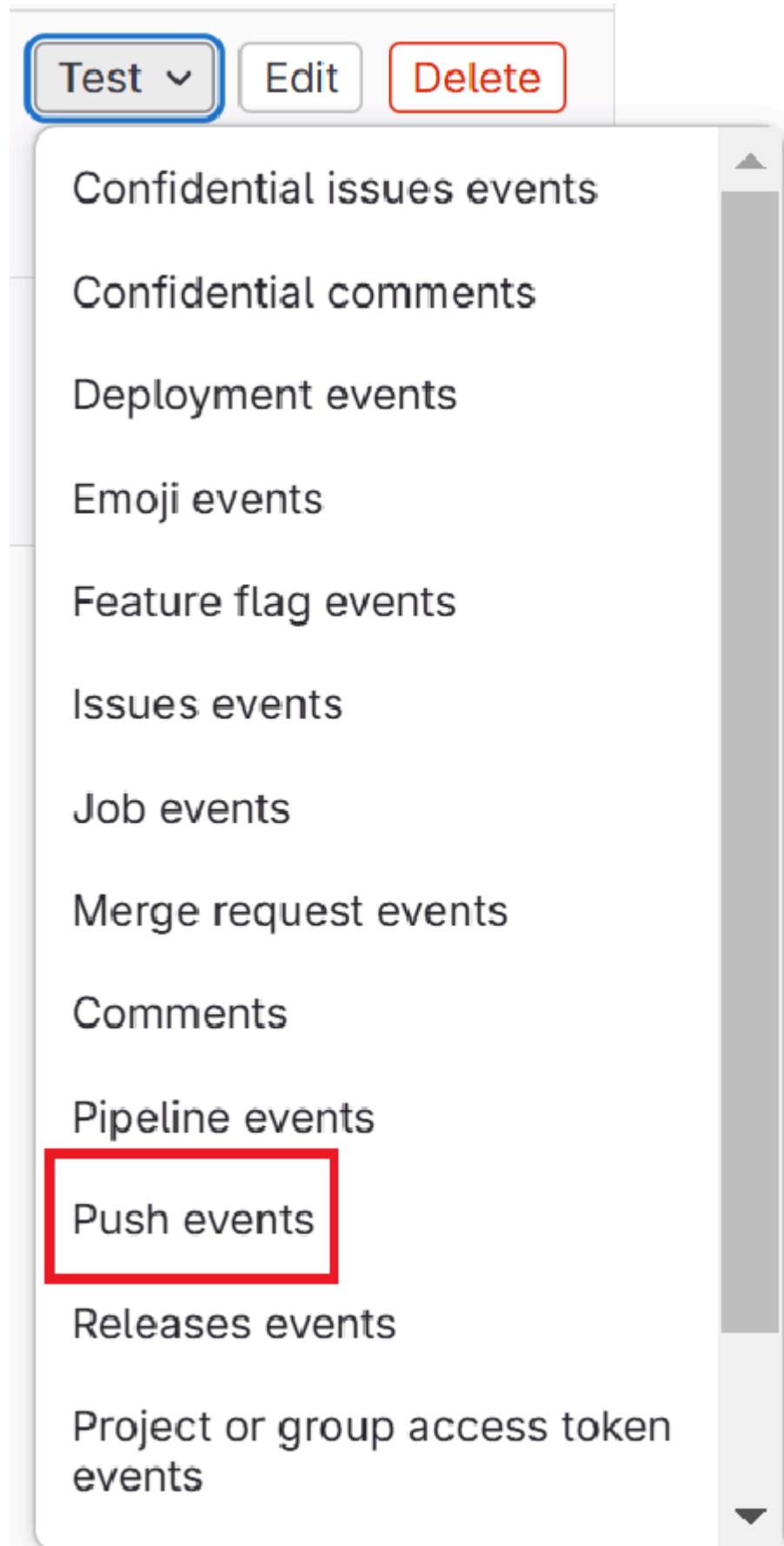
Branch Specifier (blank for 'any') ?

*/master

- 아래 체크박스를 클릭한다.



- 그리고 GitLab에서 만든 Webhook으로 들어가서 Push events 를 클릭하면 Jenkins에서 실행이 된다.



Push events를 누르면 Jenkins 컨테이너가 실행이 되어 한다.

Jenkins Step

```
#!/bin/bash

# Jenkins 환경에서 자동으로 제공되는 WORKSPACE 변수를 확인하고, 없으면 기본값
if [ -z "$WORKSPACE" ]; then
    WORKSPACE="/var/jenkins_home/workspace/Jenkins_CI_CD"
    echo "WORKSPACE 변수 설정이 없습니다. 기본값으로 설정합니다: $WORKSPACE"
else
    echo "WORKSPACE가 설정되어 있습니다: $WORKSPACE"
fi

echo "Stopping and removing old containers..."# 기존 컨테이너 종료 및 삭제
docker rm -f react spring-lb spring-rest spring-report spring-noti spring-ai || true

# 기존 React, Spring 이미지 삭제
echo "Removing old Docker images..."
docker rmi -f lil1001/react:latest lil1001/spring-lb:latest lil1001/spring-rest:latest

# ✅ 기존 node_modules 및 캐시 제거 후 재설치
echo "Clearing yarn cache and reinstalling dependencies..."
rm -rf node_modules .yarn/cache
yarn install --immutable

# React Docker 이미지 빌드 및 푸시
echo "Building React Docker image..."
cd $WORKSPACE/frontend
docker build --cache-from=lil1001/react:latest \
    --build-arg VITE_GITLAB_CLIENT_ID=8a9363db17d9a7aae6c03c37f43eec0 \
    --build-arg VITE_API_BASE=https://i12b108.p.ssafy.io/api \
    --build-arg VITE_OAUTH_REDIRECT=https://i12b108.p.ssafy.io/loading \
    -t lil1001/react:latest . # ← 반드시 `.` 추가

# docker build --no-cache --build-arg REACT_APP_PUBLIC_URL=https://i12b108.p.ssafy.io
if [ $? -ne 0 ]; then
    echo "React Docker image build failed."
```

```

    exit 1
fi
docker push lil1001/react:latest &
echo "React Docker image successfully pushed."

# Spring Boot (security) Docker 이미지 빌드 및 푸시
echo "Building Spring Boot (security) Docker image..."
chmod +x $WORKSPACE/backend/security/gradlew
cd $WORKSPACE/backend/security
./gradlew build -x test --no-daemon --parallel --build-cache
docker build --cache-from=lil1001/spring-lb:latest -t lil1001/spring-lb:latest .
if [ $? -ne 0 ]; then
    echo "Spring Boot (security) Docker image build failed."
    exit 1
fi
docker push lil1001/spring-lb:latest &
echo "Spring Boot (security) Docker image successfully pushed."

# Spring Boot (rest) Docker 이미지 빌드 및 푸시
echo "Building Spring Boot (rest) Docker image..."
cd $WORKSPACE/backend/rest
chmod +x $WORKSPACE/backend/rest/gradlew
./gradlew build -x test --no-daemon --parallel --build-cache
docker build --cache-from=lil1001/spring-rest:latest -t lil1001/spring-rest:lates
if [ $? -ne 0 ]; then
    echo "Spring Boot (rest) Docker image build failed."
    exit 1
fi
docker push lil1001/spring-rest:latest &
echo "Spring Boot (rest) Docker image successfully pushed."

# Spring Boot (report) Docker 이미지 빌드 및 푸시
echo "Building Spring Boot (report) Docker image..."
cd $WORKSPACE/backend/report
chmod +x $WORKSPACE/backend/report/gradlew
./gradlew build -x test --no-daemon --parallel --build-cache
docker build --cache-from=lil1001/spring-report:latest -t lil1001/spring-report:
if [ $? -ne 0 ]; then

```

```

    echo "Spring Boot (report) Docker image build failed."
    exit 1
fi
docker push lil1001/spring-report:latest &
echo "Spring Boot (report) Docker image successfully pushed."

# Notification Docker 이미지 빌드 및 푸시
echo "Building Notification Docker image..."
cd $WORKSPACE/backend/notification
chmod +x $WORKSPACE/backend/notification/gradlew
./gradlew build -x test --no-daemon --parallel --build-cache
docker build --cache-from=lil1001/spring-noti:latest -t lil1001/spring-noti:lates
if [ $? -ne 0 ]; then
    echo "Notification Docker image build failed."
    exit 1
fi
docker push lil1001/spring-noti:latest &
echo "Notification Docker image successfully pushed."

# AI Docker 이미지 빌드 및 푸시
echo "Building Notification Docker image..."
cd $WORKSPACE/backend/ai
chmod +x $WORKSPACE/backend/ai/gradlew
./gradlew build -x test --no-daemon --parallel --build-cache
docker build --cache-from=lil1001/spring-ai:latest -t lil1001/spring-ai:latest .
if [ $? -ne 0 ]; then
    echo "Notification Docker image build failed."
    exit 1
fi
docker push lil1001/spring-ai:latest &
echo "Notification Docker image successfully pushed."

# 새 컨테이너 실행
echo "Starting new containers..."
docker run -d --name react \
    --network my-network \
    -p 81:80 \

```

```

-v /home/ubuntu/react-nginx/conf.d/react.conf:/etc/nginx/conf.d/default.conf
lil1001/react:latest
docker run -d --name spring-lb \
-e oauth.gitlab.client-id=8a9363db17d9a7aae6c03c37f43eec0f942e30bb8
-e oauth.gitlab.client-secret=gloas-266ca321a0b27116833f4cfaa1dd90f09k
-e SPRING_DATASOURCE_URL="jdbc:mysql://i12b108.p.ssafy.io:3307/gitea
-e SPRING_DATASOURCE_USERNAME="giteat" \
-e SPRING_DATASOURCE_PASSWORD="tnrhgoTek108xla" \
--network my-network \
-p 8081:8080 \
lil1001/spring-lb:latest
docker run -d --name spring-rest \
-e oauth.gitlab.client-id=8a9363db17d9a7aae6c03c37f43eec0f942e30bb8
-e oauth.gitlab.client-secret=gloas-266ca321a0b27116833f4cfaa1dd90f09k
-e gpt.api.key=sk-proj-HeeG32y7LdXeYrZARdDjfl25z5p0Qf7EJAemplAGxz
-e SPRING_DATASOURCE_URL="jdbc:mysql://i12b108.p.ssafy.io:3307/gitea
-e SPRING_DATASOURCE_USERNAME="giteat" \
-e SPRING_DATASOURCE_PASSWORD="tnrhgoTek108xla" \
--network my-network \
-p 8082:8080 \
lil1001/spring-rest:latest
docker run -d --name spring-report \
-e "jenkins.url=http://jenkins-test:8080" \
-e "jenkins.api.token=1171c25713fe5f65086d3a2eceab21bacc" \
-e "jenkins.user=lilyoung" \
-e "jenkins.job.name=Jenkins_Lighthouse" \
-e SPRING_DATASOURCE_URL="jdbc:mysql://i12b108.p.ssafy.io:3307/gitea
-e SPRING_DATASOURCE_USERNAME="giteat" \
-e SPRING_DATASOURCE_PASSWORD="tnrhgoTek108xla" \
--network my-network \
-p 8084:8080 \
lil1001/spring-report:latest
docker run -d --name spring-noti \
-e SPRING_DATASOURCE_URL="jdbc:mysql://i12b108.p.ssafy.io:3307/gitea
-e SPRING_DATASOURCE_USERNAME="giteat" \
-e SPRING_DATASOURCE_PASSWORD="tnrhgoTek108xla" \
--network my-network \
-p 8083:8080 \

```

```

lil1001/spring-noti:latest
docker run -d --name spring-ai \
  -e SPRING_DATASOURCE_URL="jdbc:mysql://i12b108.p.ssafy.io:3307/gitea
  -e SPRING_DATASOURCE_USERNAME="giteat" \
  -e SPRING_DATASOURCE_PASSWORD="tnrhgoTek108xla" \
  -e gpt.api.key=sk-proj-HeeG32y7LdXeYrZARdDjfl25z5p0Qf7EJAemplAGxz6
  --network my-network \
  -p 8085:8080 \
  lil1001/spring-ai:latest

docker restart nginx-lb

if [ $? -eq 0 ]; then
  echo "New containers started successfully."
else
  echo "Failed to start new containers."
  exit 1
fi

# 완료
echo "Deployment completed successfully!"

```

참고 URL

<https://curiousjinan.tistory.com/entry/aws-ec2-docker-jenkins-setup>

<https://junuuu.tistory.com/443>

젠킨스 안에서 컨테이너를 띄우기 위해 아래와 같은 툴을 설치해야 한다.

젠킨스 안에 Docker 설치

(이거는 젠킨스 컨테이너를 띄울 때, EC2 에 있는 docker를 볼륨 마운트를 해서 할 필요 없을 거 같음. 한번 알아봐야 할 듯)

```

# Docker 설치
## - Old Version Remove
apt-get remove docker docker-engine docker.io containerd runc
## - Setup Repo
apt-get update
apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o \
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker
    $(lsb_release -cs) stable" | tee /etc/apt/sources.list.d/docker.list > /dev/null
## - Install Docker Engine
apt-get update
apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin

```

젠킨스 안에 node.js, npm 설치

```

apt-get update
apt-get install -y curl
curl -fsSL https://deb.nodesource.com/setup_18.x | bash -
apt-get install -y nodejs

```